C++ Midway Project: Store Inventory Management System

A menu program that stores products in a text file, keeps them in a sorted array on the heap, and finds them with a binary search.

Course: CIS-25 Programming Using C++ (L1-41736) **Student:** H Cheng **Folder:** `Midterm/`

---

1. What my program does

I wrote a program that keeps track of the products in a small online store. It is the same store app I have been building through the modules this semester, so for this project I pulled those pieces together into one program instead of starting something new.

When you run it, the first thing it does is open `inventory.txt` and read the products into an array that it creates while the program is running. Then it asks who is logging in. If you choose Admin you can add, delete and update products. If you choose Regular User you can only look at them and search. After that a menu with ten options keeps coming back until you choose Exit.

The products are always kept in order of their product ID. I did it that way on purpose, because a binary search only works if the data is sorted. When you search for an ID on option 2, the program checks the middle product and then throws away the half that cannot possibly contain what you are looking for. With 8 products that means 4 comparisons at the very worst instead of 8. I print every step so you can see it happening. I also reuse the same search when deleting a product and when checking that a new product ID is not already taken.

Option 3 is a second search that works on the descriptions instead. Those are not sorted, so that one has to check every product one at a time. Having both of them in the same program makes it easy to see why sorting the data first is worth the trouble.

Every prompt checks what you typed. Letters where a number belongs, an empty description, a negative quantity and a price of zero all get refused with a message explaining the problem, and then it asks again. When you exit, the program writes everything back to `inventory.txt`, so the next time you run it your changes are still there.

2. How I put it together

I split the program into four pairs of files so that each class has its own header and its own implementation, the same way we did it in Module 5.

```
Midterm/
  main.cpp              the menu, the switch, the input checking, the data type screen
  Product.h / .cpp      class 1, one product
  Inventory.h / .cpp    class 2, the array, the searches, the file reading and writing
  User.h / .cpp         class 3, User with Admin and RegularUser under it
  inventory.txt         the data file
```

```
inventory_report.txt   the report the program writes
output.pdf             this report
```

How to build and run it

`clang++ -Wall -Wextra -std=c++17 main.cpp Product.cpp Inventory.cpp User.cpp -o main`
`./main`

It builds with no warnings on both `clang++` and `g++`. You have to run it from inside the `Midterm` folder, otherwise it cannot find `inventory.txt`.

3. The concepts I had to use, and where they are

Concept	How I used it	Where it is
Data types and data sizes	Option 8 measures `bool`, `char`, `short`, `int`, `long`, `long long`, `float`, `double` and `std::string` with `sizeof` while the program runs. It also prints the range of an `int` and how many digits `float` and `double` can hold.	`main.cpp`, `showDataTypes()`
File types (.txt)	Two text files. `inventory.txt` holds the products and gets read at the start and written at the end. `inventory_report.txt` is a formatted report the program writes on option 7.	`inventory.txt`, `inventory_report.txt`
Use of pointers	The array is only reachable through the pointer `Product *items`. My binary search gives back a pointer to the product it found, the price update writes through that same pointer, deleting works out an index by subtracting two pointers, and option 9 walks the whole array with pointer arithmetic.	`Inventory.cpp`, `pointerWalk()`
Arrays and dynamic memory	The products live in an array I create with `new Product[capacity]`. When it runs out of room `resize()` makes a bigger one, copies everything over and frees the old one. The destructor calls `delete[]` so nothing is left behind.	`Inventory.cpp`, `constructor and resize()`
A search algorithm	`binarySearchById()` cuts the range in half every step and counts its comparisons. I reuse it for deleting a product and for checking that a new ID is not already taken. There is also a linear search over the descriptions so the two can be compared.	`Inventory.cpp`, `binarySearchById()`
Use of strings	`std::string` holds the stock codes, descriptions and usernames. I split the file lines on the `|` character, trim spaces off what the user types, convert text to upper case so searching ignores capitals, and use `find()` to check for a substring.	`Product.cpp and Inventory.cpp`
File I/O	`ifstream` reads the file one line at a time when the program starts and skips blank lines, comments and anything broken. `ofstream` writes the data file and the report. Both get closed when I am done with them.	`Inventory.cpp`, `loadFromFile() and saveToFile()`
At least two classes	I ended up with three. `Product` is one item, `Inventory` owns the array and does the searching and file work, and `User` is the base class that `Admin` and `RegularUser` inherit from.	`Product.h, Inventory.h, User.h`

Other things from earlier modules that ended up in here:

Menu and switch	Ten options inside a loop, sent to the right code by a `switch`, repeating until you choose Exit. This came from Module 2.
Input validation	Every prompt checks what you typed, explains the problem and asks again, so a typo cannot get into the data.

Encapsulation	All the data members are private. Nothing outside the class can change them except through a setter that checks the value first.
Inheritance	`Admin` and `RegularUser` both come from `User`. The menu asks the user object what it is allowed to do and the virtual function answers for whichever role it really is.
Header files	Four pairs of .h and .cpp files with include guards, compiled separately and linked together.

4. What I found hard

The dynamic array gave me the most trouble. My first version just appended every new product to the end, which was fine until I tried to search and realised the binary search was giving me wrong answers because the list was not actually sorted any more. I fixed it by finding the right slot first, shifting everything after it along by one, and only then putting the new product in. Deleting had the same problem in reverse, so I shift the other way to close the gap.

The other thing I had to be careful about was memory. Every `new[]` needs a matching `delete[]`, and when the array grows I have to copy the products across and free the old block, not just point at the new one. I also turned off copying for the Inventory class on purpose, because the default copy would have copied the pointer rather than the array and then two objects would try to free the same memory.

Picking `double` rather than `float` for the prices was a small decision that the data type screen ended up explaining for me. A `float` only holds about 6 reliable digits, which is not really enough once you are adding up a lot of money.

5. Screenshots of it running

These are from real runs of the program. What I typed shows up after each prompt.

Loading the file, logging in, and the product table

This is what you see when you run the program. It opens inventory.txt, reads 8 products into the array and says how many it got. Then it asks for a role and a username. I picked Admin here. The last line under the table shows the array capacity, which is bigger than the number of products because my array doubles itself when it fills up.

./main ~ Loading the file, logging in, and the product table (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

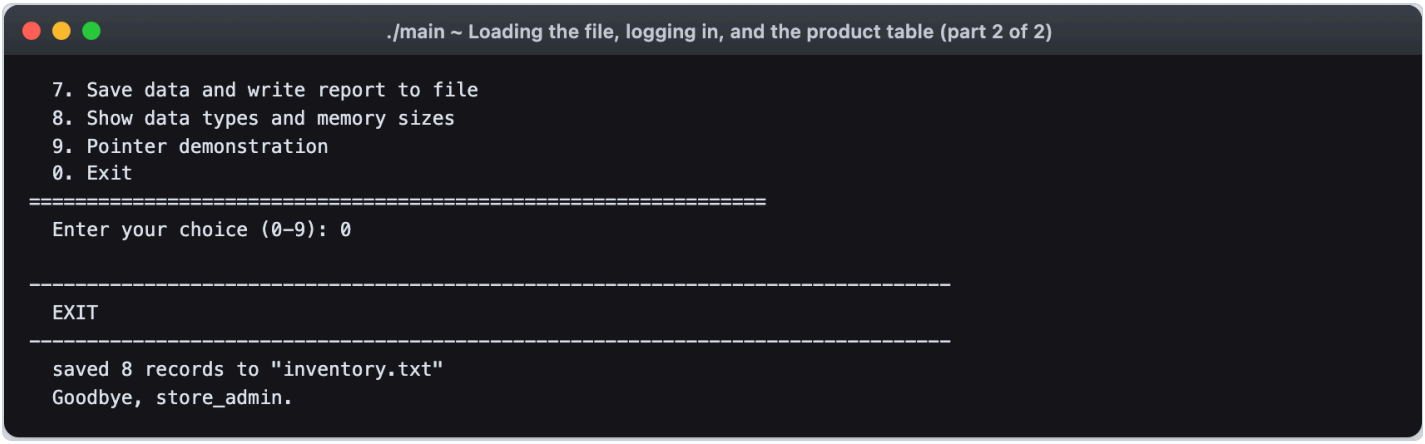
=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 1

--------------------------------------------------------------------------------
  ALL PRODUCTS (array kept sorted by product ID)
--------------------------------------------------------------------------------
    ------------------------------------------------------------------------------
    ID     STOCK     DESCRIPTION                              QTY   UNIT ($)   VALUE ($)
    ------------------------------------------------------------------------------
    105    71053     WHITE METAL LANTERN                        6       3.39       20.34
    118    84029G    KNITTED UNION FLAG HOT WATER BOTTLE        6       3.39       20.34
    150    22752     SET 7 BABUSHKA NESTING BOXES               2       7.65       15.30
    201    85123A    WHITE HANGING HEART T-LIGHT HOLDER         6       2.55       15.30
    233    22633     HAND WARMER UNION JACK                     6       1.85       11.10
    276    84029E    RED WOOLLY HOTTIE WHITE HEART.             6       3.39       20.34
    342    84406B    CREAM CUPID HEARTS COAT HANGER             8       2.75       22.00
    389    84879     ASSORTED COLOUR BIRD ORNAMENT             32       1.69       54.08
    ------------------------------------------------------------------------------
    8 records, sorted by product ID (array capacity 8)

  Total units: 72    Total value: $178.80

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
```

Loading the file, logging in, and the product table, continued (part 2 of 2)



Binary search by product ID

Looking for ID 276 only takes 2 comparisons, and proving that ID 999 is not there takes 4, out of 8 products. I made it print every step with the left, right and mid positions so you can actually watch it throw away half the list each time. It also prints how many comparisons a normal search would have needed, which is up to 8.

./main ~ Binary search by product ID (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
==============================================================
  Enter your choice (0-9): 2

--------------------------------------------------------------------------------
  BINARY SEARCH BY PRODUCT ID
--------------------------------------------------------------------------------
  Product ID to find: 276

  Searching 8 sorted records:
      step 1: left=0 right=7 mid=3 (ID 201)  -> 201 < 276, discard left half
      step 2: left=4 right=7 mid=5 (ID 276)  -> MATCH

  FOUND after 2 comparison(s):
    Product ID:   276
    Stock Code:   84029E
    Description:  RED WOOLLY HOTTIE WHITE HEART.
    Quantity:     6
    Unit Price:   $3.39
    Stock Value:  $20.34

  A linear search would need up to 8 comparisons.

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
```

Binary search by product ID, continued (part 2 of 2)

./main ~ Binary search by product ID (part 2 of 2)

```
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 2

------------------------------------------------------------------------------
  BINARY SEARCH BY PRODUCT ID
------------------------------------------------------------------------------
  Product ID to find: 999

  Searching 8 sorted records:
      step 1: left=0 right=7 mid=3 (ID 201)  -> 201 < 999, discard left half
      step 2: left=4 right=7 mid=5 (ID 276)  -> 276 < 999, discard left half
      step 3: left=6 right=7 mid=6 (ID 342)  -> 342 < 999, discard left half
      step 4: left=7 right=7 mid=7 (ID 389)  -> 389 < 999, discard left half

  NOT FOUND after 4 comparison(s).

  A linear search would need up to 8 comparisons.

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 0

------------------------------------------------------------------------------
  EXIT
------------------------------------------------------------------------------
  saved 8 records to "inventory.txt"
  Goodbye, store_admin.
```

Input validation, then adding a product

I typed the word abc where a number should go, left the description blank, put in a negative quantity and then a price of zero. The program refused all four, said what was wrong each time, and asked again instead of crashing or saving bad data. Once everything was valid it added the product in the right spot to keep the list sorted.

./main ~ Input validation, then adding a product (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

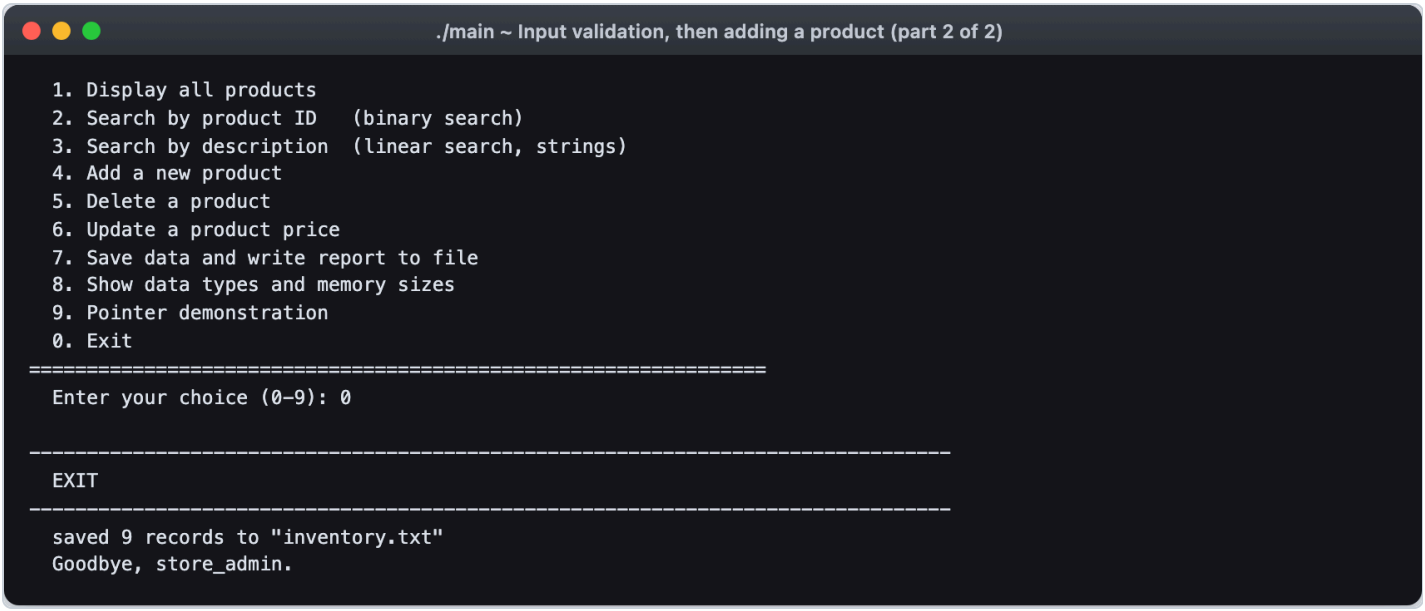
=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 4

--------------------------------------------------------------------------------
  ADD A NEW PRODUCT
--------------------------------------------------------------------------------
  Product ID (1-999999): abc
    [Invalid] That is not a whole number. Try again.
  Product ID (1-999999): 450
  Stock code:       TEST9
  Description:
    [Invalid] This cannot be left blank.
  Description:      GLASS STAR FROSTED T-LIGHT HOLDER
  Quantity (0-100000): -2
    [Invalid] Enter a number from 0 to 100000.
  Quantity (0-100000): 6
  Unit price:       $0
    [Invalid] The price must be greater than 0.00.
  Unit price:       $4.25

  Added and placed at sorted position 8 of 9.
    Product ID:   450
    Stock Code:   TEST9
    Description:  GLASS STAR FROSTED T-LIGHT HOLDER
    Quantity:     6
    Unit Price:   $4.25
    Stock Value:  $25.50

=============== MAIN MENU  (Admin: store_admin) ===============
```

Input validation, then adding a product, continued (part 2 of 2)



Searching the descriptions as text

This is my second search and it works on words instead of numbers. The descriptions are not in alphabetical order so this one has to check them one by one. I typed babushka in lower case and it still matched SET 7 BABUSHKA NESTING BOXES because I convert both sides to upper case first. The second search shows what happens when nothing matches.

./main ~ Searching the descriptions as text (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

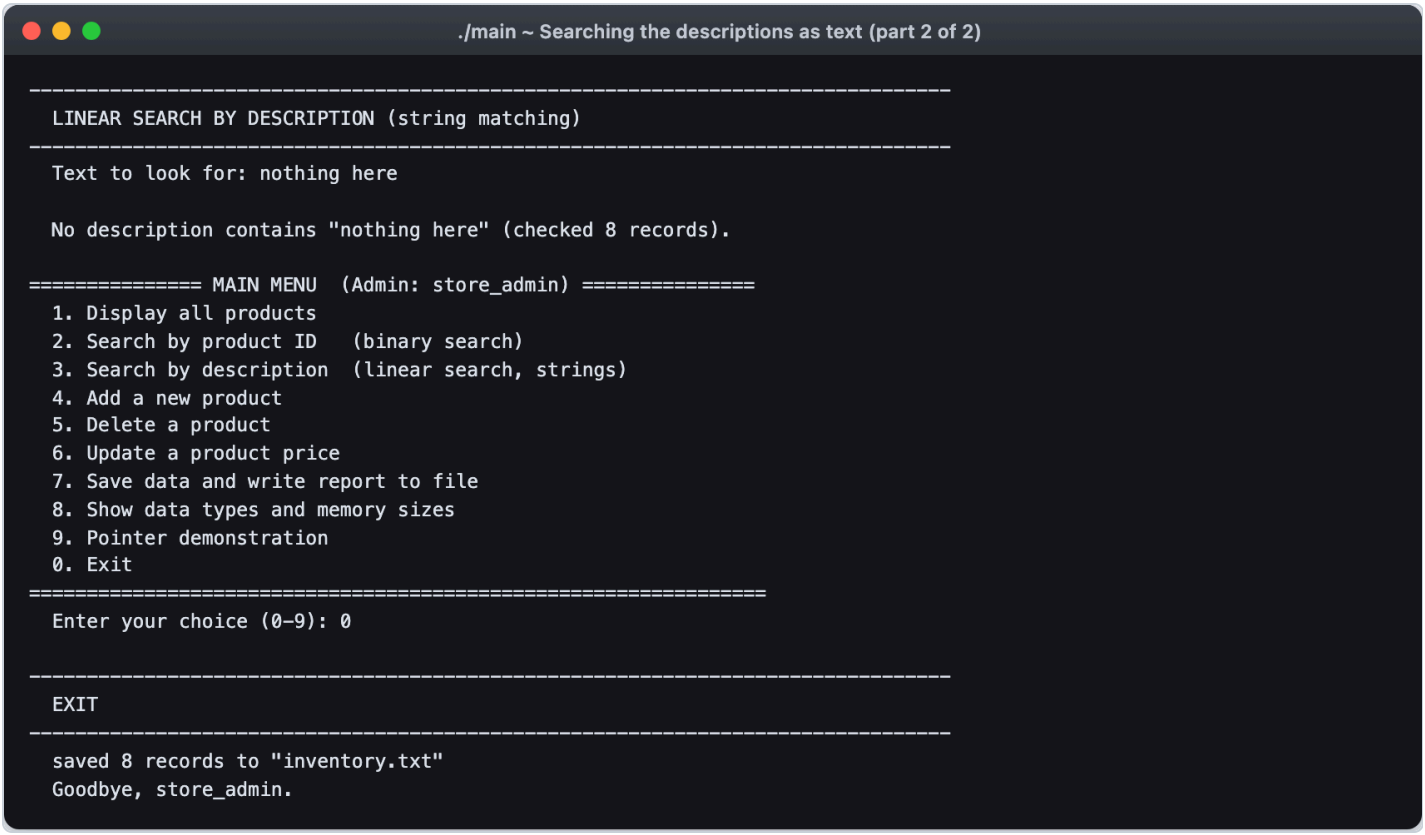
=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
==============================================================
  Enter your choice (0-9): 3

--------------------------------------------------------------------------------
  LINEAR SEARCH BY DESCRIPTION (string matching)
--------------------------------------------------------------------------------
  Text to look for: babushka

  FOUND after 3 comparison(s) (capitalisation ignored):
    Product ID:   150
    Stock Code:   22752
    Description:  SET 7 BABUSHKA NESTING BOXES
    Quantity:     2
    Unit Price:   $7.65
    Stock Value:  $15.30

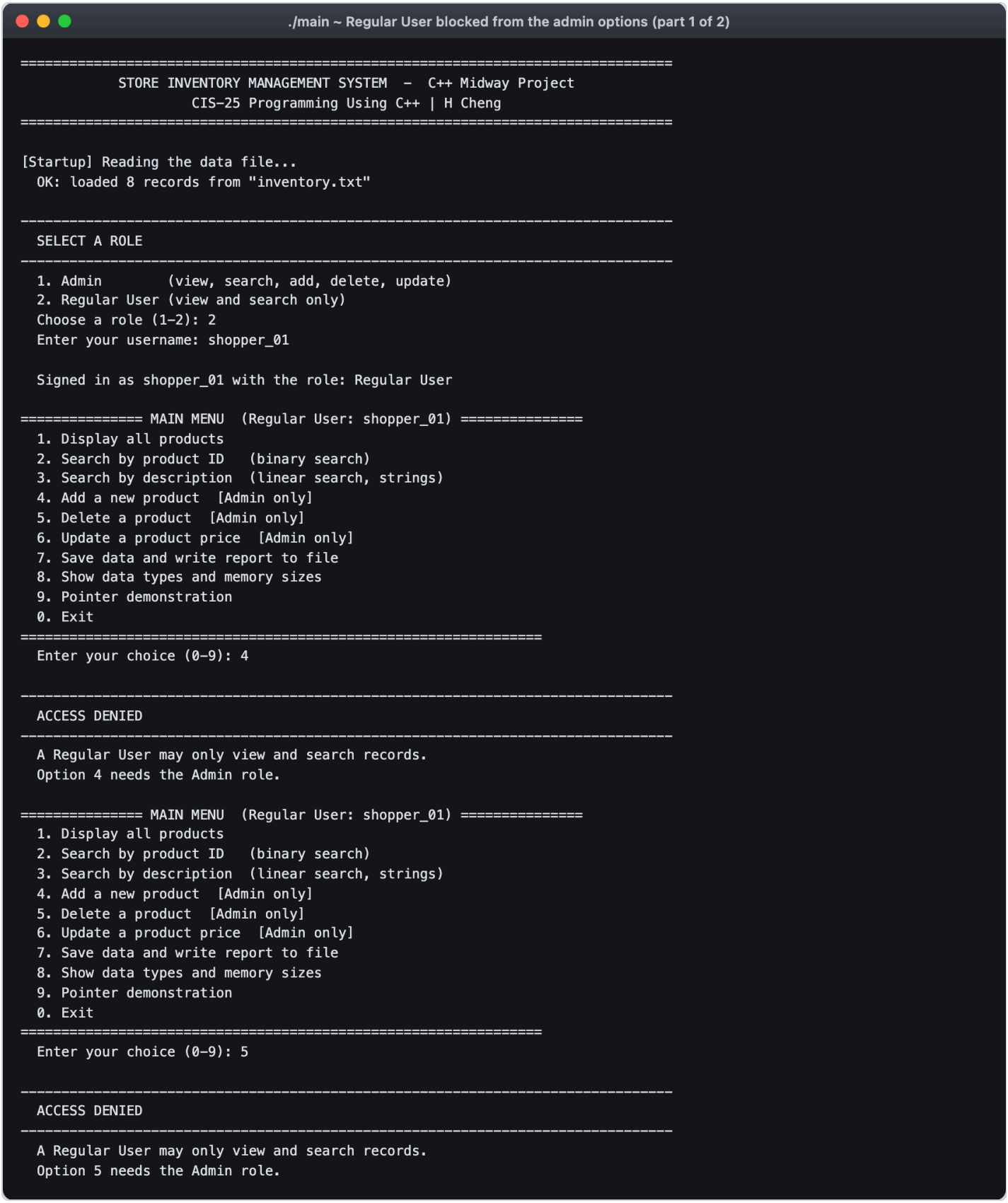
=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
==============================================================
  Enter your choice (0-9): 3
```

Searching the descriptions as text, continued (part 2 of 2)



Regular User blocked from the admin options

Here I logged in as a Regular User instead. Options 4 and 5 get refused straight away and the program says which role you would need. Viewing and searching still work fine, so the block is only on changing things, not on looking at them.



Regular User blocked from the admin options, continued (part 2 of 2)

./main ~ Regular User blocked from the admin options (part 2 of 2)

```
=============== MAIN MENU  (Regular User: shopper_01) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product  [Admin only]
  5. Delete a product  [Admin only]
  6. Update a product price  [Admin only]
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
=====================================================================
  Enter your choice (0-9): 2

-------------------------------------------------------------------------------
  BINARY SEARCH BY PRODUCT ID
-------------------------------------------------------------------------------
  Product ID to find: 342

  Searching 8 sorted records:
      step 1: left=0 right=7 mid=3 (ID 201)  -> 201 < 342, discard left half
      step 2: left=4 right=7 mid=5 (ID 276)  -> 276 < 342, discard left half
      step 3: left=6 right=7 mid=6 (ID 342)  -> MATCH

  FOUND after 3 comparison(s):
    Product ID:   342
    Stock Code:   84406B
    Description:  CREAM CUPID HEARTS COAT HANGER
    Quantity:     8
    Unit Price:   $2.75
    Stock Value:  $22.00

  A linear search would need up to 8 comparisons.

=============== MAIN MENU  (Regular User: shopper_01) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product  [Admin only]
  5. Delete a product  [Admin only]
  6. Update a product price  [Admin only]
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
=====================================================================
  Enter your choice (0-9): 0

-------------------------------------------------------------------------------
  EXIT
-------------------------------------------------------------------------------
  saved 8 records to "inventory.txt"
  Goodbye, shopper_01.
```

Updating, deleting, and writing both text files

I changed a price, deleted a product, and then used option 7 to write both files. inventory.txt is the one the program reads next time it runs, and inventory_report.txt is a tidy version for a person to read. The table at the end shows that both changes actually stuck.

./main ~ Updating, deleting, and writing both text files (part 1 of 3)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 6

--------------------------------------------------------------------------------
  UPDATE A PRODUCT PRICE
--------------------------------------------------------------------------------
  Product ID to update: 389

  Current price of ASSORTED COLOUR BIRD ORNAMENT: $1.69
  New unit price:   $1.49

  Updated through the pointer returned by the search:
    Product ID:   389
    Stock Code:   84879
    Description:  ASSORTED COLOUR BIRD ORNAMENT
    Quantity:     32
    Unit Price:   $1.49
    Stock Value:  $47.68

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
```

Updating, deleting, and writing both text files, continued (part 2 of 3)

./main ~ Updating, deleting, and writing both text files (part 2 of 3)

```
  0. Exit
================================================================
  Enter your choice (0-9): 5

--------------------------------------------------------------------------------
  DELETE A PRODUCT
--------------------------------------------------------------------------------
  Product ID to delete: 150

  Deleted product ID 150. 7 records remain.

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
================================================================
  Enter your choice (0-9): 7

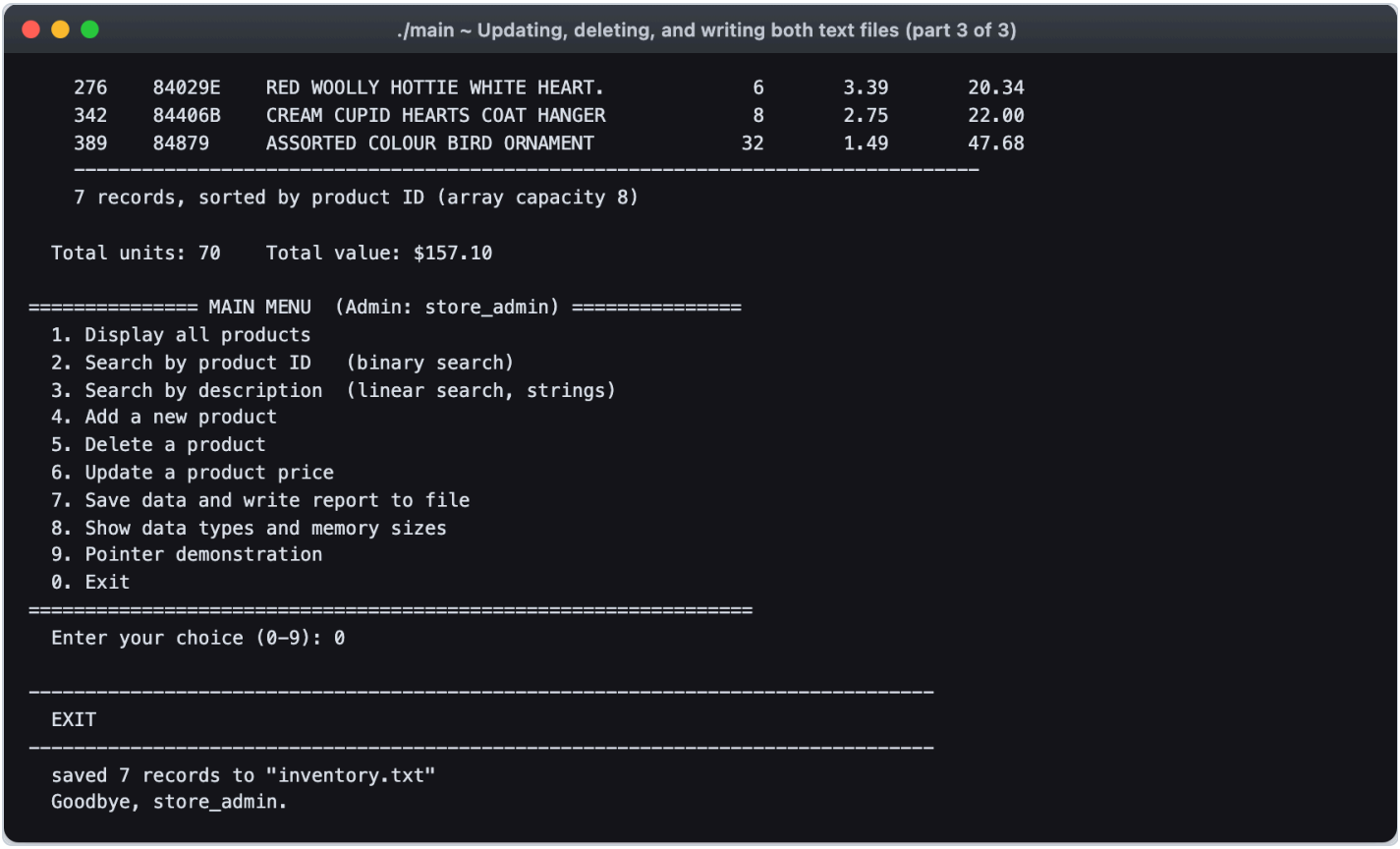
--------------------------------------------------------------------------------
  WRITE BOTH .TXT FILES
--------------------------------------------------------------------------------
  OK: saved 7 records to "inventory.txt"
  OK: wrote report to "inventory_report.txt"

  inventory.txt   holds the raw records for the next run.
  inventory_report.txt   holds a formatted report for a person to read.

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
================================================================
  Enter your choice (0-9): 1

--------------------------------------------------------------------------------
  ALL PRODUCTS (array kept sorted by product ID)
--------------------------------------------------------------------------------
    ------------------------------------------------------------------------------
    ID     STOCK     DESCRIPTION                          QTY   UNIT ($)   VALUE ($)
    ------------------------------------------------------------------------------
    105    71053     WHITE METAL LANTERN                    6       3.39       20.34
    118    84029G    KNITTED UNION FLAG HOT WATER BOTTLE    6       3.39       20.34
    201    85123A    WHITE HANGING HEART T-LIGHT HOLDER     6       2.55       15.30
    233    22633     HAND WARMER UNION JACK                 6       1.85       11.10
```

Updating, deleting, and writing both text files, continued (part 3 of 3)



Data types and how many bytes each one takes

This screen measures every type I use with sizeof while the program is running, in bytes and in bits. It also prints the range of an int and compares how many digits float and double can hold. That last bit is the reason I used double for prices instead of float.

./main ~ Data types and how many bytes each one takes (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 8

--------------------------------------------------------------------------------
  DATA TYPES AND DATA SIZES (measured with sizeof)
--------------------------------------------------------------------------------
  TYPE                  BYTES   BITS    WHAT THIS PROGRAM USES IT FOR
  ------------------------------------------------------------------------
  bool                  1       8       true/false results from setters
  char                  1       8       single characters, the | separator
  short                 2       16      (not used, shown for comparison)
  int                   4       32      productId, quantity, array indexes
  long                  8       64      (not used, shown for comparison)
  long long             8       64      (not used, shown for comparison)
  float                 4       32      (not used: too few digits for money)
  double                8       64      unitPrice, stock values, totals
  std::string           24      192     stockCode, description, usernames
  Product *             8       64      pointer to the dynamic array
  Product               72      576     one inventory record
  Inventory             16      128     pointer + count + capacity

  Ranges of the whole-number and decimal types:
    int      -2147483648 to 2147483647
    double   about 15 reliable decimal digits
    float    about 6 reliable decimal digits  <- why money uses double, not float

  Note: sizeof(std::string) is the size of the object itself. The
  characters of a long string live separately on the heap, which is
  why every description costs the same 24 bytes inside a Product.
```

Data types and how many bytes each one takes, continued (part 2 of 2)

./main ~ Data types and how many bytes each one takes (part 2 of 2)

```
=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
================================================================
  Enter your choice (0-9): 0

--------------------------------------------------------------------------------
  EXIT
--------------------------------------------------------------------------------
  saved 8 records to "inventory.txt"
  Goodbye, store_admin.
```

Walking the array with a pointer

A pointer starts at the first product and moves forward one product at a time. The addresses go up by exactly 72 bytes every step, which is the size of one Product. Then I show the same description being read three different ways that all mean the same thing, and I subtract two pointers to get back an index.

./main ~ Walking the array with a pointer (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 9

--------------------------------------------------------------------------------
  POINTER DEMONSTRATION
--------------------------------------------------------------------------------
    The array lives on the heap. 'items' holds its first address.
    sizeof(Product) = 72 bytes, so each step
    of the pointer moves that many bytes forward.

    INDEX   ADDRESS           OFFSET    ID      DESCRIPTION
    ------------------------------------------------------------
    0       0x1015cc0f0       0         105     WHITE METAL LANTERN
    1       0x1015cc138       72        118     KNITTED UNION FLAG HOT WATER BOTTLE
    2       0x1015cc180       144       150     SET 7 BABUSHKA NESTING BOXES
    3       0x1015cc1c8       216       201     WHITE HANGING HEART T-LIGHT HOLDER
    4       0x1015cc210       288       233     HAND WARMER UNION JACK
    5       0x1015cc258       360       276     RED WOOLLY HOTTIE WHITE HEART.
    6       0x1015cc2a0       432       342     CREAM CUPID HEARTS COAT HANGER
    7       0x1015cc2e8       504       389     ASSORTED COLOUR BIRD ORNAMENT

    Three ways to read the SAME first description:
      items[0].getDescription()    -> WHITE METAL LANTERN
      (*(items + 0)).getDescription() -> WHITE METAL LANTERN
      (items + 0)->getDescription()   -> WHITE METAL LANTERN

    Reading one single value through a pointer:
      pointer to last record: 0x1015cc2e8
      last->getQuantity()     = 32 units
```

Walking the array with a pointer, continued (part 2 of 2)

